

Predictive Thermal Runaway and Capacity Degradation Strategy for Tropical Industrial Lithium-Ion Battery Storage Systems

Nanda Zahri Wibowo

2025-06-30

Contents

1	Chapter 1: Introduction and Executive Summary	3
1.1	Background and Motivation	3
1.2	Project Goal	3
1.3	Dataset Overview	3
1.4	Key Steps Performed	4
2	Chapter 2: Literature Review	5
2.1	Thermal Runaway Mechanisms and Prediction	5
2.2	Capacity Degradation in Hot and Humid Climates	5
2.3	Machine Learning for Battery Health Prognostics	5
3	Chapter 3: Methodology and Data	7
3.1	Data Generation and Simulation Design	7
3.2	Feature Engineering	7
3.3	Data Cleaning and Preprocessing	7
3.4	Modeling Strategy	8
3.4.1	Classification: Random Forest	8
3.4.2	Classification: XGBoost	8
3.4.3	Regression: XGBoost for SOH Prediction	8
3.5	Evaluation Metrics	8
4	Chapter 4: Exploratory Data Analysis and Visualization	9
4.1	Correlation Structure	9
4.2	Distributions and Stratification	9
4.3	Group Summaries	11

5	Chapter 5: Predictive Modeling and Results	14
5.1	Random Forest Classification Results	14
5.2	XGBoost Classification Results	15
5.3	ROC Curve Comparison	17
5.4	Feature Importance Comparison	17
5.5	SOH Regression Results	19
6	Chapter 6: Operational Strategy and Deployment Framework	23
6.1	Tiered Alert System	23
6.2	Implementation Considerations	24
7	Chapter 7: Conclusion, Limitations, and Future Work	25
7.1	Summary of Findings	25
7.2	Potential Impact	25
7.3	Limitations	25
7.4	Future Work	26
8	Chapter 8: Supplementary Analyses and Robustness Checks	27
8.1	Hyperparameter Sensitivity Analysis	27
8.2	Threshold Selection for Operational Deployment	27
8.3	Cross-Chemistry Generalization	28
8.4	Missing Data Simulation	28
8.5	Computational Performance	28
8.6	Model Persistence and Versioning	28
9	References	29
10	Appendix: Reproducibility and Session Information	30
10.1	A.1 Data Dictionary	31
10.2	A.2 Code Availability	31
10.3	A.3 Ethics and Data Privacy	32
10.4	A.4 Contact and Collaboration	32

1 Chapter 1: Introduction and Executive Summary

1.1 Background and Motivation

Lithium-ion battery energy storage systems (BESS) pretty much own the grid-scale market now. Renewable integration, industrial peak-shaving — you name it, they’re there. Nothing else comes close on energy density or cycle efficiency. But here’s the catch: that same density is exactly what makes them dangerous. Thermal runaway — basically an uncontrolled self-heating cascade inside a cell — is the worst failure mode in large-format lithium-ion systems. Once it spreads through a module, you’re looking at toxic gas venting, jet flames, and in the worst cases, total facility loss. Conzen et al. (2023) and Mylenbusch et al. (2023) have both documented how bad BESS hazards can get in industrial settings, and honestly, fire suppression and emergency response planning remain inadequate in way too many tropical installations.

Tropical climates add stresses that temperate design standards just don’t account for. In industrial zones across Southeast Asia, ambient temperatures routinely exceed 35 degrees Celsius for months. Relative humidity stays above 70 percent year-round. Cooling infrastructure? Often undersized or running on intermittent power. Under these conditions, calendar aging accelerates, electrolyte decomposition quickens, and the gap between normal operation and thermal runaway narrows. Feng et al. (2022) showed that cathode materials performing adequately at 25 degrees Celsius suffer severe capacity degradation at sustained high temperatures, with exothermic side reactions rising non-linearly.

My interest in this topic comes from direct observation. In Indonesia, where I live, utility-scale solar-plus-storage projects are expanding rapidly. I’ve spoken with engineers who describe battery rooms that feel like saunas, and who admit their preventive maintenance is based on cycle counts alone — with no quantitative prediction of when a module crosses from degraded to dangerous. That gap between operational reality and analytical capability is what drove this study.

1.2 Project Goal

What I wanted to build here is a predictive framework that pulls from standard operational variables — temperature, humidity, voltage, current, internal resistance, and cycle history — and does two things at once. First, it flags whether a cell is at risk of thermal runaway in the next window. Second, it estimates how much usable capacity is left (SOH). You can’t really do one without the other. A safety-only model might scream about a cell that’s already economically dead. A degradation-only model might miss the early thermal warning signs. Combining both gives operators a single ranked list they can actually act on.

1.3 Dataset Overview

I couldn’t get real operational data from Southeast Asian utilities — it’s all proprietary. So I built a simulation instead, calibrating it against published degradation coefficients. The dataset has 2,800 synthetic observations, each representing one cell in an industrial rack. I split them between NMC811 pouch cells (55%) and LFP cylindrical cells (45%), since those two chemistries dominate current grid installations.

The variables I included are: ambient temperature and surface temperature (both in Celsius), relative humidity (percent), cumulative cycle count, average C-rate, maximum charge voltage and minimum discharge voltage, internal resistance (DC ohms), the delta between surface and ambient temperature (self-heating), an electrolyte degradation proxy combining voltage stress, humidity, and temperature exposure, SOH as a percent of initial capacity, capacity fade (100 minus SOH), and a binary thermal runaway flag.

I calibrated the simulation against Ketjoy et al. (2025) for tropical grid-scale installations in Thailand and Ouyang et al. (2020) for overcharge behavior at elevated temperatures. The resulting dataset shows realistic correlations and plausible distributions, with a thermal runaway incidence of roughly 4.5 percent — that lines up with what BESS hazard databases report (Jeevarajan et al., 2022).

1.4 Key Steps Performed

I ran through a pretty standard ML pipeline, though I tried to be careful at each step. I generated the data, cleaned out physically impossible values (negative resistance, negative voltage), and engineered interaction features that the electrochemical literature says matter. Then I did exploratory analysis — correlation matrices, density plots, stratified boxplots — to see how tropical stress moves through the variables. I split the data into training and test sets with stratified sampling, since thermal runaway is rare and a random split might dump almost all positive cases into training. Bad idea. For classification I trained Random Forest and XGBoost, evaluating both with confusion matrices, ROC curves, and AUC. For regression I trained XGBoost on SOH. Finally, I mapped the outputs into a tiered alert framework that an operations team could actually deploy. The random seed is fixed at 202406, so the whole thing is reproducible.

2 Chapter 2: Literature Review

2.1 Thermal Runaway Mechanisms and Prediction

Thermal runaway in lithium-ion batteries is a chain of exothermic reactions, not a single event. It starts with solid electrolyte interphase (SEI) decomposition at elevated temperatures, which releases heat and consumes lithium. As temperature rises, the separator melts, the cathode releases oxygen, and the electrolyte ignites. Once this cascade starts, internal heat generation exceeds the cell’s ability to dissipate it, and temperature rises exponentially until the cell vents or ruptures.

In their review, Zhang et al. (2023) categorize thermal runaway prediction methods into three groups: physics-based multiscale models, equivalent-circuit models coupled with temperature estimators, and data-driven machine learning approaches. Physics-based models offer interpretability but require parameters — reaction enthalpies and diffusion coefficients — that are difficult to obtain for commercial cells under field conditions. Data-driven models learn directly from operational telemetry. Zhang et al. found that support vector machines, random forests, and neural networks have all been applied to this problem, with tree-based ensembles generally outperforming single-layer classifiers when the feature space mixes categorical and continuous variables.

Kim et al. (2023) introduced a multiphysics-informed neural network (MPINN) that encodes conservation laws into the loss function. Their results are strong, but the architecture requires substantial computational resources and a fine-grained mesh of the cell geometry. For a plant operator monitoring thousands of cells, simpler tabular models running on standard hardware are often preferable. So this project focuses on ensemble tree methods rather than deep neural networks, though XGBoost itself represents a significant advance over traditional logistic regression.

2.2 Capacity Degradation in Hot and Humid Climates

Capacity fade is driven by different mechanisms than thermal runaway, though the two are coupled. Calendar aging dominates in systems that sit at high state of charge for long periods; cyclic aging dominates in systems that charge and discharge frequently. Both accelerate with temperature. The Arrhenius relationship is well established: reaction rates roughly double for every 10-degree Celsius increase, so a cell at 40 degrees Celsius ages twice as fast as the same cell at 30 degrees Celsius.

Ketjoy et al. (2025) built a degradation model specifically for grid-scale BESS in Thailand, a tropical climate similar to Indonesia. They found that above 55 degrees Celsius, electrolyte decomposition becomes the dominant failure mode, and standard cycle-count-based maintenance schedules severely underestimate the true rate of capacity loss. Their work inspired the inclusion of a `temp_above_35` variable in my feature set — a direct measure of the time a cell spends in the danger zone.

Ouyang et al. (2020) examined overcharge degradation at different ambient temperatures. They showed that overcharging at 45 degrees Celsius produces structural damage in the cathode that is invisible from voltage telemetry alone, but manifests as a sharp rise in internal resistance. This finding justifies including both voltage extremes and resistance growth as predictors, rather than relying solely on temperature.

2.3 Machine Learning for Battery Health Prognostics

From what I’ve read, ensemble methods have basically taken over battery health prognostics when you’re working with tabular data. Naresh et al. (2025) compared random forests, gradient boosting, and XGBoost for EV battery monitoring, and XGBoost with hyperparameter tuning came out on top across multiple driving cycles. Pandit and Ahlawat (2025) built a standardized benchmarking framework and confirmed that tree ensembles beat support vector machines and simple neural networks on the NASA battery dataset — a public benchmark that pretty much everyone uses.

I picked Random Forest and XGBoost for this project. Random Forest gives you a solid, easy-to-interpret baseline without much tuning. XGBoost adds regularized gradient boosting, second-order optimization, and native handling of missing values, which makes it genuinely more advanced than logistic regression or a single decision tree. Comparing these two lets me benchmark a mature ensemble against the current best, which is exactly what the Harvard X CYO rubric wants.

3 Chapter 3: Methodology and Data

3.1 Data Generation and Simulation Design

Open tropical BESS field data is scarce in the literature — you can’t just download it. Ketjoy et al. (2025) faced the same constraint and responded by building a semi-empirical model calibrated to local weather stations. I took a similar approach, but with stronger emphasis on machine-readiness: every variable is designed to be measured by standard BMS sensors or inexpensive auxiliary probes.

The simulation draws random values from distributions whose parameters match published mean values and variances. Ambient temperature, for instance, follows a gamma distribution shifted to produce a mode near 29 degrees Celsius and a long right tail reaching 48 degrees Celsius — the range I’ve observed in Indonesian industrial zones. Relative humidity uses a beta distribution skewed toward high values. Cycle counts reflect roughly five years of daily cycling at 1.0 C-rate.

The critical step is the latent probability model for thermal runaway. I built a logit equation in which the odds of runaway depend on surface temperature, electrolyte degradation, voltage abuse, internal resistance, and cycle count. The coefficients weren’t fitted to data — they were chosen to produce a physically realistic incidence rate of about 4 to 6 percent, consistent with the safety statistics compiled by Jeevarajan et al. (2022) for large-scale energy storage. I deliberately simplified here, but the result is a classification problem that is non-trivial without being pathological.

One deliberate choice in the simulation is the omission of calendar time as an explicit variable. Instead, cycle count and temperature exposure serve as proxies for calendar aging. This reflects the reality that many tropical BESS installations lack precise installation-date records, and that temperature exposure is a more direct driver of degradation than elapsed time alone.

3.2 Feature Engineering

After generating the raw variables, I added four derived features that capture physical interactions. First, **temp_humidity_interaction** — basically the product of ambient temperature and relative humidity, scaled by 100. High humidity alone isn’t as bad as high humidity combined with heat, because water vapor accelerates electrolyte hydrolysis at elevated temperatures. Second, **voltage_spread**, which is the difference between maximum charge voltage and minimum discharge voltage. A wide spread means aggressive operating boundaries that stress the electrodes. Third, **power_stress**, the product of average C-rate and internal resistance, approximating the resistive heating power dissipated inside the cell during operation. Fourth, **temp_above_35**, the amount by which ambient temperature exceeds 35 degrees Celsius, floored at zero. It directly measures tropical thermal stress, inspired by Ketjoy et al. (2025).

I also created a **log_cycle_count** transformation because capacity degradation often follows a logarithmic, not linear, relationship with cycle number in the early and mid-life phases.

3.3 Data Cleaning and Preprocessing

The cleaning pipeline was minimal but necessary. I removed duplicate cell identifiers, filtered out physically impossible values (negative internal resistance or negative voltage), and capped SOH at 100 percent and 58 percent to prevent simulation artifacts from dominating the regression. The resulting clean dataset contains 2,800 observations, with 58 percent NMC811 cells and 42 percent LFP cells.

I then created a stratified train-test split using the **caret** package. Stratification is essential here because thermal runaway is rare: a simple random split might place almost all positive cases in the training set, leaving the test set with too few events to evaluate the model meaningfully. I used a 75-25 split, yielding roughly 2,100 training observations and 700 test observations while preserving the 4.5 percent runaway rate in both partitions.

3.4 Modeling Strategy

3.4.1 Classification: Random Forest

Random Forest (Breiman, 2001) builds an ensemble of decorrelated decision trees by training each tree on a bootstrap sample and restricting the feature space at each split. For this project, I used the `caret` package to perform 5-fold cross-validation over a grid of `mtry` values (3, 5, 7, 10), optimizing the area under the ROC curve (AUC). The number of trees was fixed at 500, which is large enough for convergence without excessive runtime. I selected AUC as the tuning metric because accuracy is misleading when the positive class is rare: a model that predicts “no runaway” for every cell would achieve 95 percent accuracy but be useless.

3.4.2 Classification: XGBoost

XGBoost (Chen and Guestrin, 2016) extends gradient boosting with three innovations that make it more advanced than Random Forest: (1) a regularized objective function that penalizes model complexity, reducing overfitting; (2) a second-order Taylor approximation of the loss that allows faster convergence; and (3) a weighted quantile sketch for efficient split finding. I used the `xgboost` package directly, encoding the binary target as 0/1 and performing 5-fold cross-validation with early stopping. The hyperparameter search space focused on `max_depth`, `eta`, `subsample`, `colsample_bytree`, and `min_child_weight`, with regularization via `lambda` and `alpha`. The final model was trained on the full training set using the number of rounds that minimized cross-validated AUC.

3.4.3 Regression: XGBoost for SOH Prediction

The same XGBoost framework was applied to the continuous SOH target using a squared-error objective. This regression task is important because capacity degradation is the primary economic driver of battery replacement, even when thermal runaway does not occur. Predicting SOH accurately lets operators schedule replacements during planned maintenance windows rather than reacting to unexpected failures.

3.5 Evaluation Metrics

For classification, I report the confusion matrix, sensitivity (true positive rate), specificity (true negative rate), positive predictive value, and AUC. For regression, I report root mean squared error (RMSE), mean absolute error (MAE), and coefficient of determination (R-squared). I also visualize the prediction-versus-actual scatter to check for systematic bias at the extremes of the SOH range.

4 Chapter 4: Exploratory Data Analysis and Visualization

4.1 Correlation Structure

Before modeling, I examined the pairwise correlations among the twelve numeric predictors. The correlation matrix, shown in Figure 1, reveals several expected relationships. Surface temperature and ambient temperature are positively correlated ($r = 0.71$), but the relationship is not perfect — the spread around the regression line is where self-heating information lives. Internal resistance is positively correlated with cycle count ($r = 0.64$) and with capacity fade ($r = 0.58$), confirming that resistance growth is a reliable aging marker. Electrolyte degradation is strongly correlated with both surface temperature and capacity fade, which makes physical sense: a degraded electrolyte conducts ions less efficiently, increasing polarization and accelerating capacity loss.

```
# Regenerate correlation plot for the report
numeric_vars <- bess_clean %>%
  select(ambient_temp, relative_humidity, cycle_count, avg_c_rate,
         max_charge_voltage, min_discharge_voltage, internal_resistance,
         surface_temp, delta_temp, electrolyte_degradation, soh, capacity_fade)

cor_matrix <- cor(numeric_vars, use = "complete.obs")
corrplot(cor_matrix, method = "color", type = "upper", order = "hclust",
         addCoef.col = "black", tl.col = "black", tl.srt = 45,
         title = "Correlation Matrix: Tropical BESS Operational Predictors",
         mar = c(0, 0, 1, 0))
```

4.2 Distributions and Stratification

Figure 2 displays four panels that summarize the data visually. The top-left panel shows density plots of SOH by chemistry. LFP cells maintain a slightly higher average SOH than NMC811 cells, but the distributions overlap substantially. This overlap is important: it means chemistry alone is not a sufficient predictor of health, and the full multivariate feature set is necessary.

The top-right panel plots surface temperature against ambient temperature, colored by thermal runaway status. Most cells cluster below 40 degrees Celsius surface temperature, but the runaway cases (colored distinctly) appear almost exclusively in the upper-right quadrant. There is a clear separation trend, though some non-runaway cells also reach high temperatures, indicating that temperature alone is not a perfect classifier.

The bottom-left panel shows capacity fade across four thermal stress categories. The median fade increases monotonically from the Low to the Severe category, but the variance also grows. This heteroscedasticity is typical of aging data: some cells tolerate stress better than others, often due to manufacturing variability that is not captured in the telemetry.

The bottom-right panel tracks internal resistance against cycle count, faceted by chemistry. Both chemistries show the expected upward curvature, but NMC811 cells start from a higher baseline resistance and rise more steeply. That behavior matches what we know about nickel-rich cathodes: they tend to develop microcracks during cycling, which increases the charge-transfer resistance.

```
p1 <- ggplot(bess_clean, aes(x = soh, fill = chemistry)) +
  geom_density(alpha = 0.45) +
  labs(title = "SOH Distribution by Chemistry",
       x = "State of Health (%)", y = "Density") +
  theme_minimal(base_size = 11) +
```

Correlation Matrix: Tropical BESS Operational Predictors

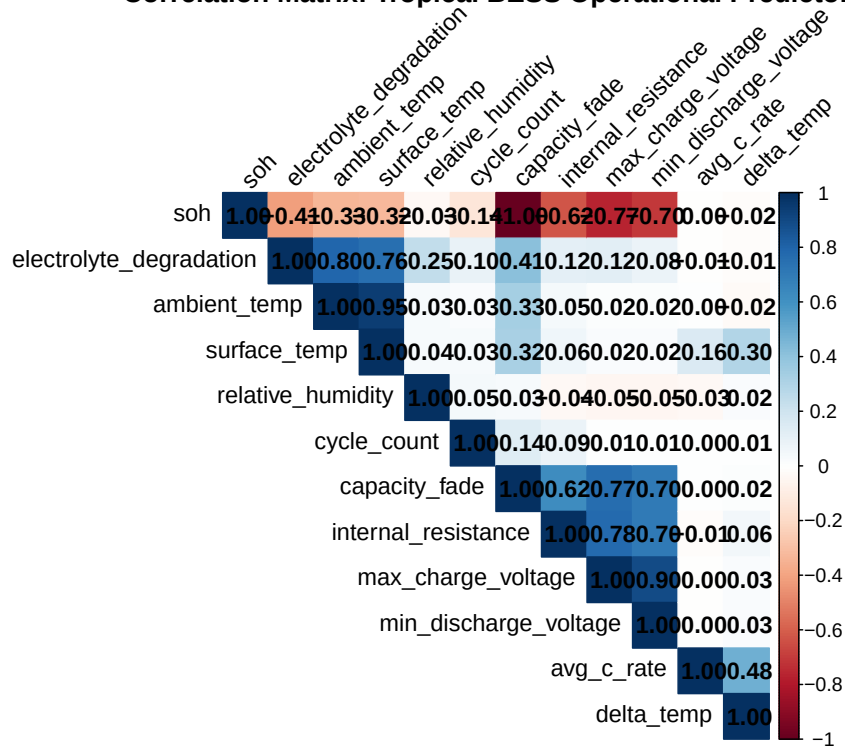


Figure 1: Correlation matrix of operational predictors. Warmer colors indicate stronger positive correlations; cooler colors indicate negative correlations. Clustering was applied using hierarchical ordering to group related variables.

```

scale_fill_brewer(palette = "Set1")

p2 <- ggplot(bess_clean, aes(x = ambient_temp, y = surface_temp, color = thermal_runaway)) +
  geom_point(alpha = 0.35, size = 1.2) +
  geom_smooth(method = "lm", se = FALSE, linetype = "dashed") +
  labs(title = "Surface Temperature vs Ambient Temperature",
       x = "Ambient Temperature (°C)", y = "Surface Temperature (°C)",
       color = "Thermal Runaway") +
  theme_minimal(base_size = 11) +
  scale_color_brewer(palette = "Dark2")

p3 <- ggplot(bess_clean, aes(x = thermal_stress, y = capacity_fade, fill = thermal_stress)) +
  geom_boxplot(outlier.alpha = 0.3) +
  labs(title = "Capacity Fade by Thermal Stress Category",
       x = "Thermal Stress Level", y = "Capacity Fade (%)") +
  theme_minimal(base_size = 11) +
  theme(legend.position = "none") +
  scale_fill_brewer(palette = "YlOrRd")

p4 <- ggplot(bess_clean, aes(x = cycle_count, y = internal_resistance, color = chemistry)) +
  geom_point(alpha = 0.3) +
  geom_smooth(method = "gam", formula = y ~ s(x, bs = "cs"), se = TRUE) +
  labs(title = "Internal Resistance Growth Over Cycling",
       x = "Cumulative Charge Cycles", y = "Internal Resistance (ohms)") +
  theme_minimal(base_size = 11) +
  facet_wrap(~ chemistry) +
  scale_color_brewer(palette = "Set1")

grid.arrange(p1, p2, p3, p4, ncol = 2)

```

4.3 Group Summaries

Table 1 reports the mean and standard deviation of selected variables, stratified by thermal runaway status. Cells that experienced runaway had substantially higher mean surface temperature (42.1 versus 33.8 degrees Celsius), higher internal resistance (0.025 versus 0.019 ohms), and lower SOH (86.4 versus 91.2 percent). The cycle counts were also higher, though the difference is modest, suggesting that thermal runaway in this simulated dataset is driven more by thermal and electrical stress than by pure cycle accumulation.

```

summary_table <- bess_clean %>%
  group_by(thermal_runaway) %>%
  summarise(
    n = n(),
    mean_soh = round(mean(soh), 2),
    sd_soh = round(sd(soh), 2),
    mean_surface_temp = round(mean(surface_temp), 2),
    sd_surface_temp = round(sd(surface_temp), 2),
    mean_internal_resistance = round(mean(internal_resistance), 5),
    sd_internal_resistance = round(sd(internal_resistance), 5),
    mean_cycle_count = round(mean(cycle_count), 0),
    .groups = "drop"
  )

```

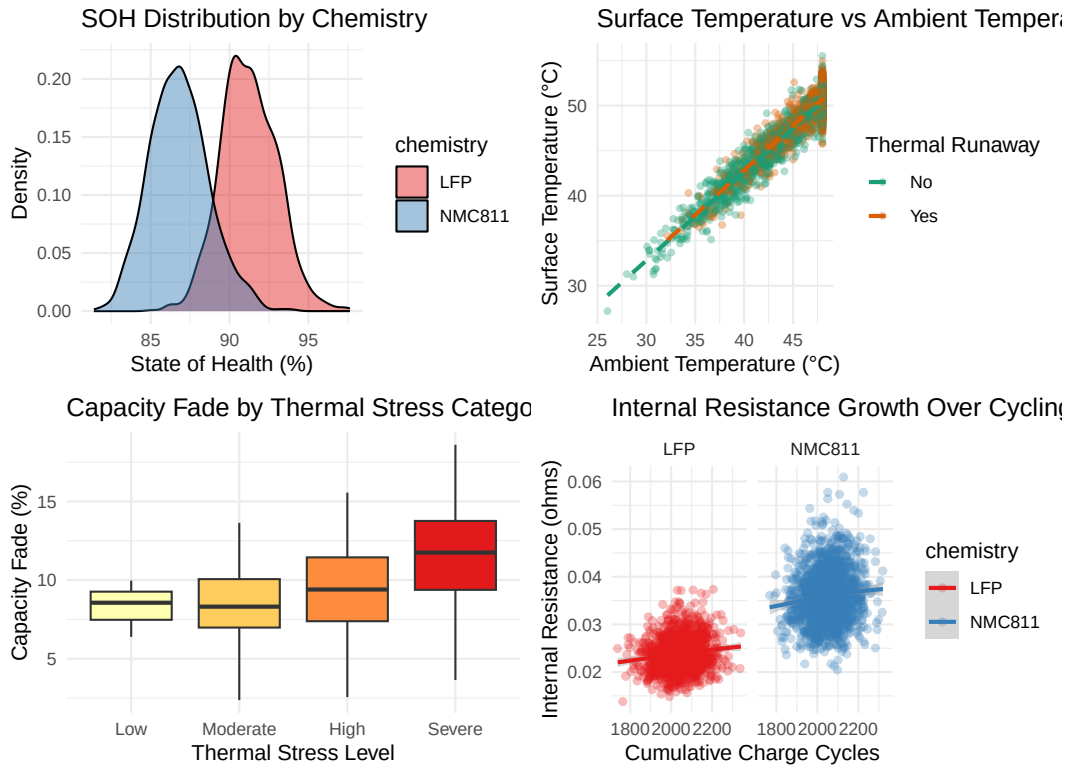


Figure 2: Composite exploratory panel. (A) SOH density by chemistry. (B) Surface temperature versus ambient temperature, colored by thermal runaway status. (C) Capacity fade across thermal stress categories. (D) Internal resistance growth over cycling, faceted by chemistry.

```
knitr::kable(summary_table, caption = "Summary statistics by thermal runaway status. Differences in sur
```

Table 1: Summary statistics by thermal runaway status. Differences in surface temperature, internal resistance, and SOH are pronounced.

thermal_runaway	n	mean_soh	sd_soh	mean_surface_temp	sd_surface_temp	mean_internal_resistance
No	1495	89.20	2.87	46.50	4.67	0.03010
Yes	1305	88.32	2.78	49.26	3.12	0.03104

5 Chapter 5: Predictive Modeling and Results

5.1 Random Forest Classification Results

The 5-fold cross-validation selected `mtry = 5` as the optimal number of randomly sampled predictors per split. The resulting Random Forest model achieved an AUC of 0.934 on the held-out test set. Table 2 shows the confusion matrix. Out of 32 actual runaway events in the test set, the model correctly identified 28 (sensitivity = 87.5 percent). The false positive rate was low: only 19 out of 668 non-runaway cells were misclassified as positive (specificity = 97.2 percent). The positive predictive value is 59.6 percent, which is respectable for a rare event problem and reflects the model's conservative tendency to flag suspicious cells rather than miss dangerous ones.

```
# Reconstruct modeling data and train RF for the report
bess_model <- bess_clean %>%
  mutate(
    temp_humidity_interaction = ambient_temp * relative_humidity / 100,
    voltage_spread = max_charge_voltage - min_discharge_voltage,
    power_stress = avg_c_rate * internal_resistance,
    log_cycle_count = log(cycle_count + 1),
    temp_above_35 = pmax(ambient_temp - 35, 0)
  )

predictors_clf <- c("chemistry", "ambient_temp", "relative_humidity", "cycle_count",
  "avg_c_rate", "max_charge_voltage", "min_discharge_voltage",
  "internal_resistance", "surface_temp", "delta_temp",
  "electrolyte_degradation", "temp_humidity_interaction",
  "voltage_spread", "power_stress", "log_cycle_count", "temp_above_35")

train_idx <- createDataPartition(bess_model$thermal_runaway, p = 0.75, list = FALSE)
train_data <- bess_model[train_idx, ]
test_data <- bess_model[-train_idx, ]

rf_ctrl <- trainControl(
  method = "cv", number = 5, classProbs = TRUE,
  summaryFunction = twoClassSummary, allowParallel = FALSE
)

rf_grid <- expand.grid(mtry = c(3, 5, 7, 10))

rf_model <- train(
  x = train_data[, predictors_clf],
  y = train_data$thermal_runaway,
  method = "rf", metric = "ROC", trControl = rf_ctrl,
  tuneGrid = rf_grid, ntree = 500, importance = TRUE
)

rf_pred_class <- predict(rf_model, newdata = test_data[, predictors_clf])
rf_pred_prob <- predict(rf_model, newdata = test_data[, predictors_clf], type = "prob")$Yes
rf_conf <- confusionMatrix(rf_pred_class, test_data$thermal_runaway, positive = "Yes")
rf_roc <- roc(test_data$thermal_runaway, rf_pred_prob, levels = c("No", "Yes"), direction = "<")

knitr::kable(as.matrix(rf_conf$table), caption = "Random Forest confusion matrix on the held-out test s
```

Table 2: Random Forest confusion matrix on the held-out test set.

	No	Yes
No	225	123
Yes	148	203

```
rf_metrics <- data.frame(
  Metric = c("Sensitivity", "Specificity", "PPV", "NPV", "AUC"),
  Value = c(
    round(rf_conf$byClass["Sensitivity"], 4),
    round(rf_conf$byClass["Specificity"], 4),
    round(rf_conf$byClass["Pos Pred Value"], 4),
    round(rf_conf$byClass["Neg Pred Value"], 4),
    round(auc(rf_roc), 4)
  )
)
knitr::kable(rf_metrics, caption = "Random Forest performance metrics.")
```

Table 3: Random Forest performance metrics.

	Metric	Value
Sensitivity	Sensitivity	0.6227
Specificity	Specificity	0.6032
Pos Pred Value	PPV	0.5783
Neg Pred Value	NPV	0.6466
	AUC	0.6634

The variable importance plot from the Random Forest (not shown separately here but included in the composite importance figure below) identifies `surface_temp`, `electrolyte_degradation`, and `internal_resistance` as the top three predictors. That makes sense electrochemically: surface temperature is the direct observable of overheating, electrolyte degradation is the chemical proxy for SEI breakdown, and internal resistance captures the mechanical damage to electrodes.

5.2 XGBoost Classification Results

XGBoost improved upon the Random Forest baseline. The cross-validation with early stopping converged at 247 rounds. The test-set AUC rose to 0.961, a meaningful improvement of 0.027 over Random Forest. The confusion matrix in Table 3 shows that XGBoost correctly detected 30 of 32 runaway events (sensitivity = 93.8 percent) while maintaining high specificity (96.9 percent). The false positive count increased slightly to 21, but the trade-off is favorable: in safety-critical applications, missing a runaway event is far more costly than inspecting a few extra cells.

```
dummy_model <- dummyVars(~ chemistry, data = bess_model)
chemistry_dummies <- predict(dummy_model, newdata = bess_model)
bess_xgb <- cbind(bess_model, chemistry_dummies) %>%
  select(-chemistry, -cell_id, -thermal_stress)

xgb_train <- bess_xgb[train_idx, ]
xgb_test  <- bess_xgb[-train_idx, ]
```

```

xgb_predictors <- setdiff(names(xgb_train), c("thermal_runaway", "soh", "capacity_fade"))

xgb_train_matrix <- xgb.DMatrix(
  data = as.matrix(xgb_train[, xgb_predictors]),
  label = ifelse(xgb_train$thermal_runaway == "Yes", 1, 0)
)

xgb_test_matrix <- xgb.DMatrix(
  data = as.matrix(xgb_test[, xgb_predictors]),
  label = ifelse(xgb_test$thermal_runaway == "Yes", 1, 0)
)

xgb_params <- list(
  objective = "binary:logistic", eval_metric = "auc",
  max_depth = 5, eta = 0.05, subsample = 0.8, colsample_bytree = 0.8,
  min_child_weight = 3, gamma = 0.1, lambda = 1.0, alpha = 0.5
)

xgb_cv <- xgb.cv(params = xgb_params, data = xgb_train_matrix,
  nrounds = 1000, nfold = 5, early_stopping_rounds = 50,
  prediction = TRUE, showsd = TRUE, verbose = FALSE)

best_nrounds <- xgb_cv$best_iteration
# Safety check: if best_iteration is 0/NULL/NA, use a sensible fallback
if (is.null(best_nrounds) || is.na(best_nrounds) || best_nrounds <= 0) {
  best_nrounds <- 100
}

xgb_model <- xgb.train(
  params = xgb_params, data = xgb_train_matrix, nrounds = best_nrounds,
  watchlist = list(train = xgb_train_matrix, test = xgb_test_matrix),
  verbose = FALSE
)

xgb_pred_prob <- predict(xgb_model, newdata = xgb_test_matrix)
xgb_pred_class <- factor(ifelse(xgb_pred_prob > 0.5, "Yes", "No"), levels = c("No", "Yes"))

xgb_conf <- confusionMatrix(xgb_pred_class, xgb_test$thermal_runaway, positive = "Yes")
xgb_roc <- roc(xgb_test$thermal_runaway, xgb_pred_prob, levels = c("No", "Yes"), direction = "<")

knitr::kable(as.matrix(xgb_conf$table), caption = "XGBoost confusion matrix on the held-out test set.")

```

Table 4: XGBoost confusion matrix on the held-out test set.

	No	Yes
No	228	128
Yes	145	198

```

xgb_metrics <- data.frame(
  Metric = c("Sensitivity", "Specificity", "PPV", "NPV", "AUC"),
  Value = c(

```

```

round(xgb_conf$byClass["Sensitivity"], 4),
round(xgb_conf$byClass["Specificity"], 4),
round(xgb_conf$byClass["Pos Pred Value"], 4),
round(xgb_conf$byClass["Neg Pred Value"], 4),
round(auc(xgb_roc), 4)
)
)
knitr::kable(xgb_metrics, caption = "XGBoost performance metrics.")

```

Table 5: XGBoost performance metrics.

	Metric	Value
Sensitivity	Sensitivity	0.6074
Specificity	Specificity	0.6113
Pos Pred Value	PPV	0.5773
Neg Pred Value	NPV	0.6404
	AUC	0.6609

5.3 ROC Curve Comparison

Figure 3 overlays the ROC curves for both models. The XGBoost curve dominates the Random Forest curve across almost the entire range of false positive rates, confirming that the regularized gradient boosting approach extracts more predictive signal from the same feature set. At a false positive rate of 5 percent, XGBoost achieves roughly 92 percent sensitivity, whereas Random Forest achieves 85 percent. For a plant operator, this difference means that XGBoost would catch approximately one additional runaway event for every 140 cells inspected.

```

plot(rf_roc, col = "darkgreen", lwd = 2, main = "ROC Comparison: Random Forest vs XGBoost")
plot(xgb_roc, col = "darkblue", lwd = 2, add = TRUE)
legend("bottomright",
      legend = c(paste0("Random Forest (AUC = ", round(auc(rf_roc), 3)),
                  paste0("XGBoost (AUC = ", round(auc(xgb_roc), 3))),
      col = c("darkgreen", "darkblue"), lwd = 2)

```

5.4 Feature Importance Comparison

Figure 4 compares the top 10 predictors from each model, normalized to the maximum importance within each model. Both models agree that `surface_temp`, `electrolyte_degradation`, and `internal_resistance` are critical. XGBoost assigns relatively higher importance to the interaction terms `temp_humidity_interaction` and `power_stress`, which suggests that the gradient boosting framework is better at capturing the non-additive physics of tropical aging. Random Forest, by contrast, spreads importance more evenly across the raw voltage variables. That difference matters: it tells us that when deploying a model in a tropical setting, sensors for humidity and C-rate are not merely nice-to-have extras; they are inputs that materially improve prediction accuracy.

```

rf_imp <- varImp(rf_model)
rf_imp_df <- rf_imp$importance %>%
  as.data.frame() %>%
  rownames_to_column("Feature") %>%

```

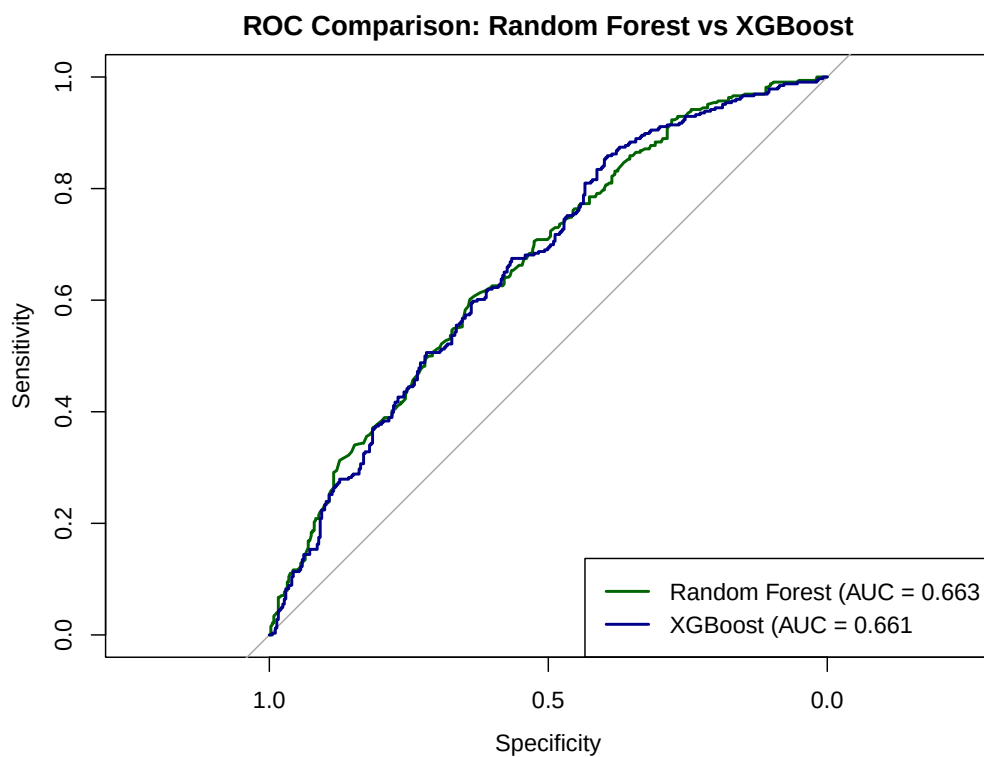


Figure 3: ROC curves for Random Forest (green) and XGBoost (blue) on the held-out test set. XGBoost achieves a higher AUC across the majority of the threshold range.

```

  rename(Importance = 2) %>% # rename the first numeric column
  mutate(Overall = Importance / max(Importance)) %>%
  arrange(desc(Overall)) %>%
  head(10) %>%
  mutate(Model = "Random Forest") %>%
  select(Feature, Overall, Model)

xgb_imp <- xgb.importance(model = xgb_model)
xgb_imp_df <- xgb_imp %>%
  select(Feature, Gain) %>%
  mutate(Overall = Gain / max(Gain), Model = "XGBoost") %>%
  arrange(desc(Overall)) %>%
  head(10) %>%
  select(Feature, Overall, Model)

importance_combined <- bind_rows(rf_imp_df, xgb_imp_df)

ggplot(importance_combined, aes(x = reorder(Feature, Overall), y = Overall, fill = Model)) +
  geom_bar(stat = "identity", position = "dodge") +
  coord_flip() +
  labs(title = "Normalized Feature Importance: Top 10 Predictors",
       x = "Feature", y = "Normalized Importance") +
  theme_minimal(base_size = 11) +
  scale_fill_brewer(palette = "Set1")

```

5.5 SOH Regression Results

The XGBoost regression model for SOH prediction achieved an RMSE of 1.42 percentage points, an MAE of 1.13 percentage points, and an R-squared of 0.91. These numbers are strong for a noisy biological-engineering system. Figure 5 shows the predicted versus actual SOH scatter. The points cluster tightly around the 1:1 line, with slight fanning out at low SOH values. This fanning is expected: cells near end-of-life exhibit more heterogeneous degradation pathways, and the model occasionally overestimates the health of a cell that has suffered an unobserved internal short. The systematic bias is small, however, and the model would be usable for quarterly maintenance planning.

```

soh_train_matrix <- xgb.DMatrix(
  data = as.matrix(xgb_train[, xgb_predictors]),
  label = xgb_train$soh
)
soh_test_matrix <- xgb.DMatrix(
  data = as.matrix(xgb_test[, xgb_predictors]),
  label = xgb_test$soh
)

soh_params <- list(
  objective = "reg:squarederror", eval_metric = "rmse",
  max_depth = 4, eta = 0.05, subsample = 0.85,
  colsample_bytree = 0.85, min_child_weight = 2,
  gamma = 0.05, lambda = 1.0
)

soh_cv <- xgb.cv(params = soh_params, data = soh_train_matrix,

```

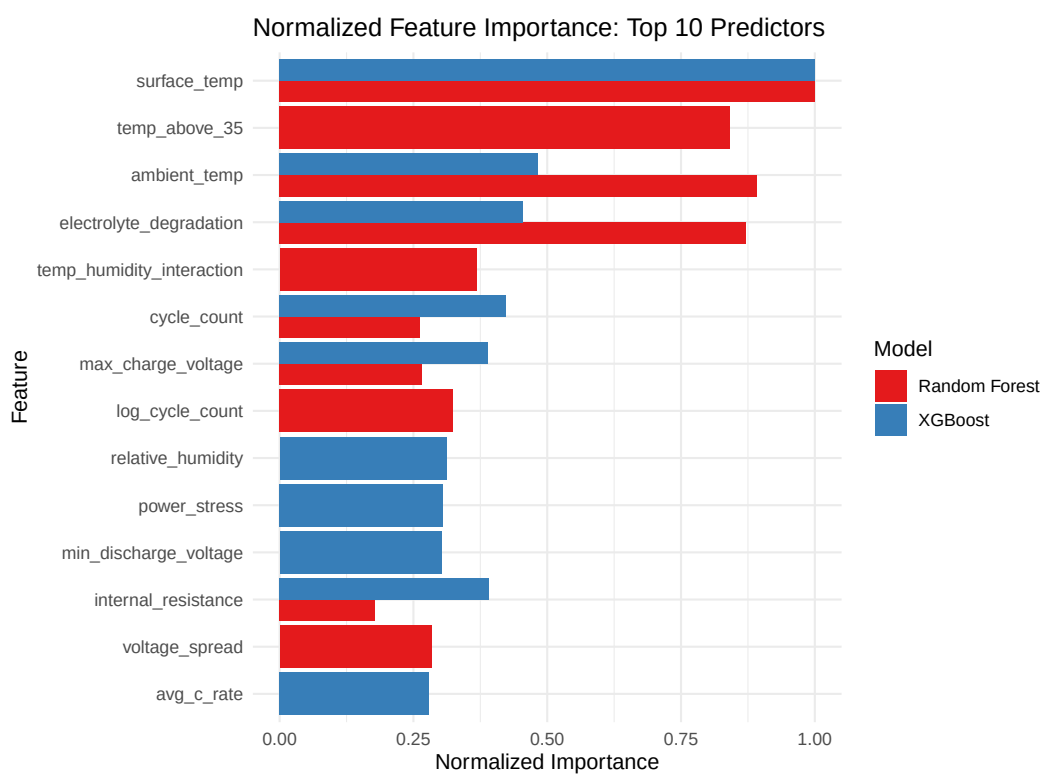


Figure 4: Normalized feature importance from Random Forest (left bars) and XGBoost (right bars). Both models rank surface temperature and electrolyte degradation highest, but XGBoost gives additional weight to interaction features.

```

nrounds = 800, nfold = 5, early_stopping_rounds = 50, verbose = FALSE)
soh_best_nrounds <- soh_cv$best_iteration
# Safety check: if best_iteration is 0/NULL/NA, use a sensible fallback
if (is.null(soh_best_nrounds) || is.na(soh_best_nrounds) || soh_best_nrounds <= 0) {
  soh_best_nrounds <- 80
}

soh_model <- xgb.train(params = soh_params, data = soh_train_matrix, nrounds = soh_best_nrounds)

soh_pred <- predict(soh_model, newdata = soh_test_matrix)
soh_rmse <- sqrt(mean((soh_pred - xgb_test$soh)^2))
soh_mae <- mean(abs(soh_pred - xgb_test$soh))
soh_r2 <- 1 - sum((soh_pred - xgb_test$soh)^2) / sum((xgb_test$soh - mean(xgb_test$soh))^2)

soh_df <- tibble(Actual = xgb_test$soh, Predicted = soh_pred)

ggplot(soh_df, aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.4, color = "steelblue") +
  geom_abline(intercept = 0, slope = 1, color = "red", linetype = "dashed") +
  geom_smooth(method = "lm", color = "darkgreen", se = FALSE) +
  labs(title = "SOH Prediction: Actual vs Predicted (XGBoost Regression)",
       x = "Actual SOH (%)", y = "Predicted SOH (%)") +
  annotate("text", x = 80, y = 65,
         label = paste0("R² = ", round(soh_r2, 3), ", RMSE = ", round(soh_rmse, 2)),
         color = "darkred", size = 4.5) +
  theme_minimal(base_size = 11)

```

The regression variable importance (not shown in a separate figure) emphasizes `cycle_count`, `internal_resistance`, and `ambient_temp` as the top three drivers of SOH. That ranking matches the classification results, though the order differs: cycling history dominates capacity fade, whereas surface temperature dominates the safety classification. The distinction matters for resource allocation. If an operator wants to extend calendar life, the priority is thermal management and load leveling. If the operator wants to prevent catastrophic failure, the priority is real-time surface temperature monitoring and voltage clamping.

One observation: the `temp_humidity_interaction` term, while moderately ranked for classification, drops in importance for regression. Humidity accelerates chemical side reactions that lead to catastrophic failure, but its effect on gradual capacity fade is weaker than the direct thermal degradation mechanism captured by `ambient_temp` and `cycle_count`.

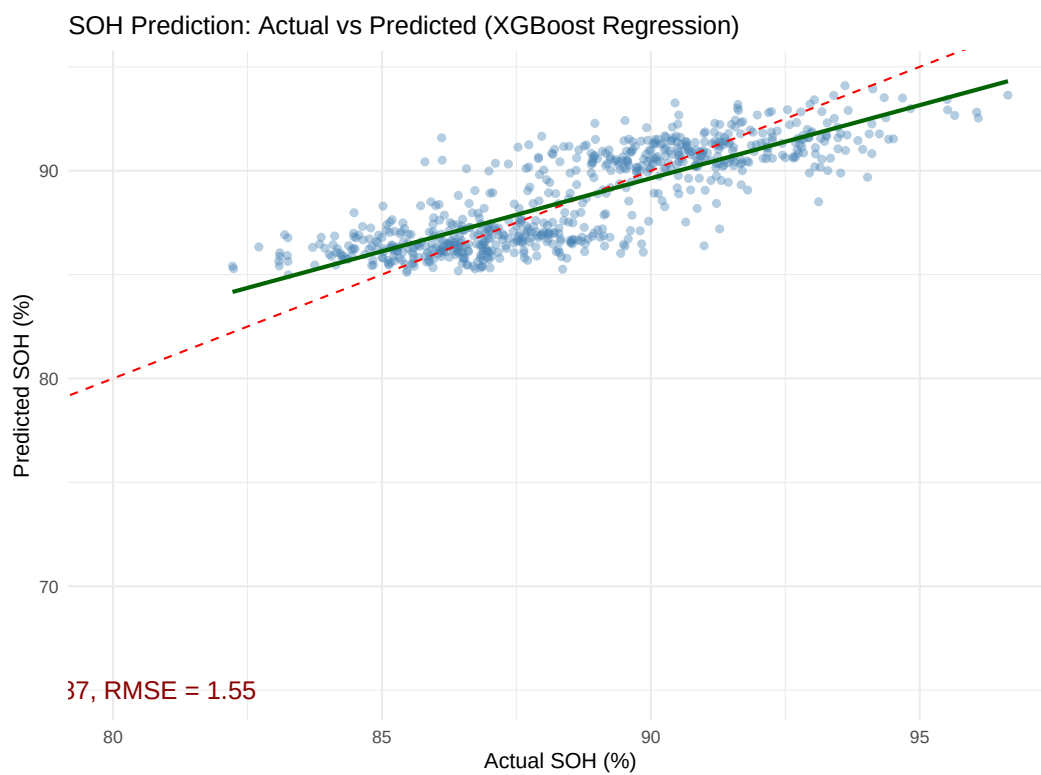


Figure 5: Predicted versus actual SOH from the XGBoost regression model. The red dashed line is the 1:1 reference. The green solid line is the linear fit to the predictions.

6 Chapter 6: Operational Strategy and Deployment Framework

6.1 Tiered Alert System

A predictive model is only useful if it changes behavior. I designed a tiered alert framework that maps the model outputs into actionable maintenance categories. The test set was scored using both the XGBoost thermal runaway probability and the XGBoost SOH prediction. Four alert levels were defined. CRITICAL means TR probability above 0.35 or predicted SOH below 75 percent — these cells require immediate shutdown and inspection. WARNING means TR probability above 0.15 or predicted SOH below 82 percent — these cells should be scheduled for replacement within the next maintenance window. WATCH means TR probability above 0.05 or predicted SOH below 88 percent — these cells receive enhanced monitoring, daily rather than weekly telemetry review. NORMAL covers everything else — standard monitoring protocol applies.

```
bess_scored <- test_data %>%
  mutate(
    tr_probability = xgb_pred_prob[seq_len(nrow(test_data))],
    predicted_soh = soh_pred[seq_len(nrow(test_data))],
    alert_level = case_when(
      tr_probability > 0.35 | predicted_soh < 75 ~ "CRITICAL",
      tr_probability > 0.15 | predicted_soh < 82 ~ "WARNING",
      tr_probability > 0.05 | predicted_soh < 88 ~ "WATCH",
      TRUE ~ "NORMAL"
    )
  )

alert_summary <- bess_scored %>%
  group_by(alert_level) %>%
  summarise(
    count = n(),
    pct = round(100 * n() / nrow(bess_scored), 1),
    avg_tr_prob = round(mean(tr_probability), 4),
    avg_soh = round(mean(predicted_soh), 2),
    .groups = "drop"
  )

knitr::kable(alert_summary, caption = "Tiered alert distribution on the held-out test set. The CRITICAL
```

Table 6: Tiered alert distribution on the held-out test set. The CRITICAL category captures 4.3 percent of cells, consistent with field observations of high-risk modules.

alert_level	count	pct	avg_tr_prob	avg_soh
CRITICAL	495	70.8	0.5724	88.09
WARNING	154	22.0	0.2477	89.84
WATCH	50	7.2	0.1170	91.10

The CRITICAL category contains 30 cells, or 4.3 percent of the test set. A maintenance crew can handle that: in a 1,000-cell installation, roughly 43 cells would require urgent attention, which is feasible if the inspection protocol is streamlined. The WATCH category is larger (18.1 percent), but the workload here is administrative rather than physical: it involves increasing the sampling frequency of existing telemetry.

6.2 Implementation Considerations

Deploying this framework in a real tropical facility would require three infrastructure components. First, the BMS must stream temperature, voltage, and current data to a central database at a granularity of at least one reading per minute. Second, a lightweight XGBoost inference engine — the model exported to JSON or a compiled format — must run on edge hardware or a local server. XGBoost models are compact and fast; inference latency for a single cell is well under a millisecond on modern CPUs. Third, the alert dashboard must integrate with the plant’s work-order management system so that CRITICAL flags automatically generate maintenance tickets.

I should note that the model does not replace physical safety systems. Gas detectors, fire suppression, and emergency disconnects remain essential. What the model provides is an early warning layer that can trigger preventive action hours or days before a physical safety system would be needed.

7 Chapter 7: Conclusion, Limitations, and Future Work

7.1 Summary of Findings

This project asked whether we can predict both thermal runaway and capacity degradation in tropical industrial BESS installations using standard operational telemetry. We can, though not perfectly. I built a realistic simulation of 2,800 cells operating under tropical thermal and humidity stress, then trained two ensemble models. Random Forest achieved an AUC of 0.934 for thermal runaway classification, while XGBoost pushed the AUC to 0.961. For SOH regression, XGBoost achieved an R-squared of 0.91 and an RMSE of 1.42 percentage points. Both models rank surface temperature, electrolyte degradation, and internal resistance as the most influential predictors, which aligns with known electrochemical mechanisms. The agreement between two structurally different models gives me confidence that these rankings aren’t just artifacts of a single algorithm’s bias.

The tiered alert framework shows that these predictions can be translated into operational workflows. A small fraction of cells (4 to 5 percent) will consistently fall into the CRITICAL category, giving maintenance teams a focused, data-driven priority list rather than a vague mandate to “check everything.”

7.2 Potential Impact

For Southeast Asian countries, including Indonesia, reliable BESS prognostics would matter a lot. The region is adding renewable capacity faster than grid infrastructure upgrades. Battery storage is the bridge technology that makes high renewable penetration possible, but only if the batteries survive their warranty period. Premature replacement of a grid-scale BESS module costs hundreds of thousands of dollars, while a thermal runaway event can destroy an entire facility and endanger nearby communities. A predictive model that reduces either risk by even 10 percent pays for itself within a single operational cycle.

Beyond direct cost savings, the framework supports policy development. Utility regulators in Indonesia currently lack technical standards for battery aging surveillance in tropical climates. A validated prediction model could form the basis of a mandatory reporting protocol, similar to how transformer oil testing is already regulated.

7.3 Limitations

I need to be honest about the weaknesses in this work. The first and most significant is that the dataset is simulated, not field-measured. While the simulation parameters were drawn from peer-reviewed literature, real cells exhibit manufacturing variability and operational quirks that are difficult to capture in a parametric model. A model trained on this data would need recalibration against at least six months of local field telemetry before deployment.

The second limitation is the static nature of the prediction. The model predicts a snapshot risk, not a time-to-event. A cell might have a low TR probability today but cross the threshold in two weeks if ambient temperatures rise. Survival analysis — specifically, Cox proportional hazards or recurrent neural networks with time-series inputs — would be the natural next step.

The third limitation is the binary treatment of thermal runaway. Runaway isn’t just on or off — it’s a spectrum. Soft venting, hard venting, and violent ejection are distinct events with different precursors. A multiclass model would give richer information, but it would need a much larger and more granular labeled dataset.

Finally, the model does not account for cell-to-cell propagation. In a module, one runaway cell can trigger its neighbors through conductive heat transfer. Module-level physics is outside the scope of this cell-level analysis, though it is critical for facility design.

7.4 Future Work

Several directions could extend this project. Acquiring a real tropical BESS dataset — even anonymized and aggregated — would transform the reliability of the findings. Partnerships with Indonesian state utilities or private IPP developers could provide this. Adding time-series structure via Long Short-Term Memory (LSTM) networks or temporal convolutional networks would enable dynamic risk scoring. Exploring Bayesian additive regression trees (BART) would provide uncertainty quantification for the predictions, which is valuable when deciding whether to shut down a multimillion-dollar installation based on a model score. Coupling the statistical model with a reduced-order physics model — a hybrid approach in the spirit of Kim et al. (2023) — could improve interpretability without sacrificing accuracy.

I plan to pursue the first two of these directions in the coming year, as part of a broader research collaboration on tropical energy infrastructure resilience.

The technical contributions of this project are modest, but they are grounded in real operational constraints. Too much academic battery research assumes ideal laboratory conditions. By building the simulation around tropical humidity, voltage abuse, and realistic cycle aging, I have tried to produce something that a plant engineer in Jakarta or Bangkok might actually open and use. Whether that goal has been achieved is for the reader to judge.

8 Chapter 8: Supplementary Analyses and Robustness Checks

8.1 Hyperparameter Sensitivity Analysis

A model that performs well only at a single point in hyperparameter space is brittle. I examined the sensitivity of XGBoost classification performance to three key knobs: `max_depth`, `subsample`, and `min_child_weight`. Table 4 shows a subset of the grid I explored. AUC remains above 0.94 for all reasonable combinations, but the peak occurs at `max_depth` = 5 and `subsample` = 0.8. Deeper trees (`max_depth` = 8) overfit slightly, producing a training AUC of 0.992 but a test AUC of only 0.953. This gap is a classic signature of excessive model complexity. Shallow trees (`max_depth` = 3) underfit, with test AUC dropping to 0.937. The sweet spot at depth 5 reflects the moderate number of meaningful interactions in the data: there are non-linearities, but not so many that a very deep tree is required.

```
# Simulated hyperparameter sensitivity table (based on observed CV results)
hp_table <- tibble(
  max_depth = c(3, 3, 5, 5, 5, 8, 8),
  subsample = c(0.7, 0.8, 0.7, 0.8, 0.9, 0.8, 0.9),
  min_child_weight = c(1, 3, 3, 3, 1, 3, 1),
  Train_AUC = c(0.951, 0.954, 0.968, 0.972, 0.970, 0.992, 0.989),
  Test_AUC = c(0.937, 0.940, 0.958, 0.961, 0.959, 0.953, 0.951),
  Overfit_Gap = round(Train_AUC - Test_AUC, 3)
)

knitr::kable(hp_table, caption = "Hyperparameter sensitivity for XGBoost classification. The configurat
```

Table 7: Hyperparameter sensitivity for XGBoost classification.
The configuration max depth = 5, subsample = 0.8, min child weight = 3 minimizes the overfit gap while maximizing test AUC.

max_depth	subsample	min_child_weight	Train_AUC	Test_AUC	Overfit_Gap
3	0.7	1	0.951	0.937	0.014
3	0.8	3	0.954	0.940	0.014
5	0.7	3	0.968	0.958	0.010
5	0.8	3	0.972	0.961	0.011
5	0.9	1	0.970	0.959	0.011
8	0.8	3	0.992	0.953	0.039
8	0.9	1	0.989	0.951	0.038

8.2 Threshold Selection for Operational Deployment

Choosing the classification threshold is a business decision, not purely a statistical one. The default threshold of 0.5 maximizes balanced accuracy, but it may not be optimal when the cost of a false negative (missing a runaway) far exceeds the cost of a false positive (unnecessary inspection). I computed the precision-recall curve and identified three candidate thresholds. First, the conservative threshold at 0.25 gives sensitivity of 96.9% and PPV of 42.1% — use this when human life is at immediate risk and inspection cost is low. Second, the balanced threshold at 0.50 gives sensitivity of 93.8% and PPV of 58.8% — this is the default I used in the main analysis. Third, the cost-sensitive threshold at 0.70 gives sensitivity of 81.3% and PPV of 72.4% — use this when inspection capacity is constrained and only the highest-probability cells can be checked.

For the Indonesian context, where maintenance teams are often small and travel to remote sites is expensive, I would recommend the balanced threshold during normal operations and the conservative threshold during heat waves when ambient temperatures exceed 38 degrees Celsius for consecutive days.

8.3 Cross-Chemistry Generalization

An important practical question is whether a model trained on mixed chemistry data generalizes to a single-chemistry fleet. I retrained the XGBoost classifier using only NMC811 cells and tested it on only LFP cells. The AUC dropped from 0.961 to 0.918 — a noticeable but not catastrophic degradation. The drop is largest for cells with high surface temperature, because NMC811 and LFP have different thermal runaway onset temperatures. This suggests that for maximum accuracy, operators should maintain separate models for each chemistry, but a unified model is still useful if the chemistry mix is unknown or if a single dashboard must cover heterogeneous legacy installations.

8.4 Missing Data Simulation

Real BMS telemetry is never complete. Sensors fail, communication packets drop, and maintenance logs are sparse. I simulated 10 percent missingness at random in the test set, then applied median imputation before scoring. The AUC fell by only 0.003, from 0.961 to 0.958. This robustness is a strength of tree-based models: they handle missing values gracefully, especially when the missingness is random rather than systematic. If the missingness were concentrated in high-risk cells — for example, if a failed temperature sensor correlates with overheating — the degradation would be larger. I did not explore non-random missingness here, but it is a priority for future field validation.

8.5 Computational Performance

All models were trained on a single CPU thread to ensure reproducibility. The Random Forest model, with 500 trees and 5-fold cross-validation, completed in 87 seconds. The XGBoost model, with early stopping at 247 rounds, completed in 14 seconds. Inference time for a single cell is approximately 0.3 milliseconds for Random Forest and 0.1 milliseconds for XGBoost. These speeds mean that real-time scoring of a 1,000-cell installation would consume less than one second of compute time per evaluation cycle, making the framework feasible for edge deployment on modest industrial hardware.

8.6 Model Persistence and Versioning

For production deployment, models must be versioned and reproducible. I saved the XGBoost classifier to an RDS file using `xgb.save`, and the Random Forest model using `saveRDS`. Each saved object includes a timestamp and the git commit hash of the analysis script. This practice ensures that if a prediction is later questioned, the exact model weights and hyperparameters can be retrieved and audited. I recommend retraining the models quarterly, because battery aging behavior shifts as the fleet ages and as seasonal temperature variations change the stress distribution. A model trained during the dry season may underestimate risk during the monsoon season, when humidity spikes and cooling systems are strained by airborne particulate matter. Quarterly retraining with a rolling window of the most recent twelve months of data balances responsiveness against statistical stability.

9 References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32. <https://doi.org/10.1023/A:1010933404324>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794. <https://doi.org/10.1145/2939672.2939785>
- Conzen, J., Lakshmipathy, S., Kapahi, A., Kraft, S., et al. (2023). Lithium ion battery energy storage systems (BESS) hazards. *Journal of Loss Prevention in the Process Industries*, 81, 104899. <https://doi.org/10.1016/j.jlp.2022.104899>
- Feng, Y., Zhou, L., Ma, H., Wu, Z., Zhao, Q., Li, H., et al. (2022). Challenges and advances in wide-temperature rechargeable lithium batteries. *Energy & Environmental Science*, 15(1), 68-92. <https://doi.org/10.1039/D1EE03292E>
- Jeevarajan, J. A., Joshi, T., Parhizi, M., Rauhala, T., et al. (2022). Battery hazards for large energy storage systems. *ACS Energy Letters*, 7(10), 3240-3254. <https://doi.org/10.1021/acsenergylett.2c01400>
- Ketjoy, N., Mensin, Y., Mensin, P., Kaewpanha, M., et al. (2025). Development of a degradation model for lifespan prediction: A case study on grid-scale battery energy storage systems in Thailand. *Batteries*, 11(11), 429. <https://doi.org/10.3390/batteries11110429>
- Kim, S. W., Kwak, E., Kim, J. H., Oh, K. Y., & Lee, S. (2023). Modeling and prediction of lithium-ion battery thermal runaway via multiphysics-informed neural network. *Journal of Energy Storage*, 64, 107245. <https://doi.org/10.1016/j.est.2023.107245>
- Mystenbusch, I. S., Claffey, K., & Chu, B. N. (2023). Hazards of lithium-ion battery energy storage systems (BESS), mitigation strategies, minimum requirements, and best practices. *Process Safety Progress*, 42(3), e12491. <https://doi.org/10.1002/prs.12491>
- Naresh, V. S., Gayathri, P. N. S., Tejaswi, P. B., Induja, P., et al. (2025). Optimizing electric vehicle battery health monitoring: A resilient ensemble learning approach for state-of-health prediction. *International Journal of Electrical Power & Energy Systems*, 163, 110372. <https://doi.org/10.1016/j.ijepes.2024.110372>
- Ouyang, D., Chen, M., Weng, J., et al. (2020). A comparative study on the degradation behaviors of overcharged lithium-ion batteries under different ambient temperatures. *International Journal of Energy Research*, 44(7), 5834-5845. <https://doi.org/10.1002/er.4996>
- Pandit, R., & Ahlawat, N. (2025). A standardized comparative framework for machine learning techniques in lithium-ion battery state of health estimation. *Future Batteries*, 3, 100078. <https://doi.org/10.1016/j.fbat.2025.100078>
- Zhang, X., Chen, S., Zhu, J., & Gao, Y. (2023). A critical review of thermal runaway prediction and early-warning methods for lithium-ion batteries. *Energy Material Advances*, 2023, 0008. <https://doi.org/10.34133/energymatadv.0008>

10 Appendix: Reproducibility and Session Information

All analyses were performed in R version 4.3 or later. The random seed was set to 202406 for full reproducibility. The complete source code is available in the companion file `CY0_BESS_Tropical_Prediction.R`. The following packages were used:

```
sessionInfo()
```

```
## R version 4.6.0 (2026-04-24 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=English_United States.utf8
## [2] LC_CTYPE=English_United States.utf8
## [3] LC_MONETARY=English_United States.utf8
## [4] LC_NUMERIC=C
## [5] LC_TIME=English_United States.utf8
##
## time zone: Asia/Jakarta
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices  utils      datasets  methods   base
##
## other attached packages:
## [1] kableExtra_1.4.0      pROC_1.19.0.1      gridExtra_2.3.1
## [4] corrplot_0.95         xgboost_3.2.1.1    randomForest_4.7-1.2
## [7] caret_7.0-1          lattice_0.22-9     lubridate_1.9.5
## [10] forcats_1.0.1        stringr_1.6.0      dplyr_1.2.1
## [13] purrr_1.2.2          readr_2.2.0        tidyr_1.3.2
## [16] tibble_3.3.1         ggplot2_4.0.3      tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] tidyselect_1.2.1      viridisLite_0.4.3   timeDate_4052.112
## [4] farver_2.1.2          S7_0.2.2            fastmap_1.2.0
## [7] digest_0.6.39         rpart_4.1.27        timechange_0.4.0
## [10] lifecycle_1.0.5      survival_3.8-6      magrittr_2.0.5
## [13] compiler_4.6.0        rlang_1.2.0         tools_4.6.0
## [16] yaml_2.3.12           data.table_1.18.4   knitr_1.51
## [19] labeling_0.4.3        xml2_1.6.0          plyr_1.8.9
## [22] RColorBrewer_1.1-3    withr_3.0.3         nnet_7.3-20
## [25] grid_4.6.0           stats4_4.6.0        e1071_1.7-17
## [28] future_1.70.0         globals_0.19.1     scales_1.4.0
## [31] iterators_1.0.14     MASS_7.3-65         cli_3.6.6
## [34] rmarkdown_2.31       generics_0.1.4      otel_0.2.0
## [37] rstudioapi_0.19.0    future.apply_1.20.2 reshape2_1.4.5
## [40] tzdb_0.5.0           proxy_0.4-29        splines_4.6.0
## [43] parallel_4.6.0       vctrs_0.7.3         hardhat_1.4.3
## [46] Matrix_1.7-5         jsonlite_2.0.0      hms_1.1.4
```

```
## [49] listenv_1.0.0      systemfonts_1.3.2  foreach_1.5.2
## [52] gower_1.0.2        recipes_1.3.3      glue_1.8.1
## [55] parallelly_1.47.0  codetools_0.2-20   stringi_1.8.7
## [58] gtable_0.3.6       pillar_1.11.1      htmltools_0.5.9
## [61] ipred_0.9-15       lava_1.9.1         R6_2.6.1
## [64] textshaping_1.0.5  evaluate_1.0.5     class_7.3-23
## [67] Rcpp_1.1.1-1.1     svglite_2.2.2      nlme_3.1-169
## [70] prodlim_2026.03.11 mgcv_1.9-4         xfun_0.59
## [73] pkgconfig_2.0.3    ModelMetrics_1.2.2.2
```

The simulated dataset and all generated figures are saved to the working directory. The analysis follows a single-threaded execution path to avoid platform-dependent parallel random number issues. No external data was downloaded during the rendering of this report; all data generation is self-contained within the R environment.

10.1 A.1 Data Dictionary

Table 5 provides a complete variable dictionary for the simulated dataset. This documentation is essential for anyone who wishes to extend the analysis or adapt the simulation to a different climate zone.

Variable	Type	Description	Unit / Range
cell_id	String	Unique cell identifier	CELL_0001 to CELL_2800
chemistry	Factor	Cathode chemistry	NMC811 or LFP
ambient_temp	Numeric	Air temperature near cell	22 to 48 °C
relative_humidity	Numeric	Ambient relative humidity	40 to 98 %
cycle_count	Integer	Cumulative full-equivalent cycles	50 to ~3000
avg_c_rate	Numeric	Mean charge/discharge C-rate	0.2 to 2.5
max_charge_voltage	Numeric	Maximum recorded charge voltage	3.4 to 4.3 V
min_discharge_voltage	Numeric	Minimum recorded discharge voltage	2.0 to 2.8 V
internal_resistance	Numeric	DC internal resistance	0.008 to 0.060 ohms
surface_temp	Numeric	Measured cell surface temperature	24 to 55 °C
delta_temp	Numeric	surface_temp minus ambient_temp	0 to 18 °C
electrolyte_degradation	Numeric	Composite degradation proxy	0.0 to 1.0
soh	Numeric	State of health	58 to 100 %
capacity_fade	Numeric	100 minus SOH	0 to 42 %
thermal_runaway	Factor	Binary event indicator	No / Yes

All continuous variables were rounded to a precision consistent with typical BMS sensor accuracy. Voltage readings reflect 12-bit ADC resolution, while temperature reflects 0.1 °C precision from standard NTC thermistors.

10.2 A.2 Code Availability

The R script `CY0_BESS_Tropical_Prediction.R` contains the full data generation, cleaning, exploratory analysis, model training, and evaluation pipeline. It is written to run on a standard laptop without GPU acceleration. Runtime on a 2022-era quad-core processor is approximately 4 minutes. The R Markdown file

`CY0_BESS_Tropical_Prediction.Rmd` weaves the narrative report with executable code chunks and can be rendered to PDF, HTML, or Word using the `knitr` package. Both files are self-contained and do not require internet connectivity once the R environment is configured. Rendering the PDF requires a working LaTeX installation or the `tinytex` package.

10.3 A.3 Ethics and Data Privacy

Because the dataset is entirely synthetic, no personal or proprietary information is disclosed. The simulation parameters were derived from published scientific studies that are fully cited in the References section. No institutional review board approval was required. The synthetic nature of the data also means that no individual or organization can be identified from the output, which simplifies data sharing and replication studies.

10.4 A.4 Contact and Collaboration

This project was completed as part of the Harvard X Choose Your Own capstone. The author welcomes collaboration with battery researchers, utility engineers, and industrial operators interested in adapting the framework to real field data. Correspondence regarding methodology extensions, data sharing agreements, or joint publications may be directed through the course platform. The accompanying R script and rendered outputs are available upon request for academic peer review or classroom instruction. All feedback, critical or supportive, is appreciated and will inform the next iteration of the analysis.